ROBOT FRAMEWORK FOR BEGINNERS · PART 1

Set up your first project

How Robot Framework works, VS Code setup, and a Sauce Demo test suite with the Browser library (Playwright)

`python` · `venv` · `robotframework-browser` · `RobotCode`

1 · WHAT ROBOT FRAMEWORK IS

Readable keywords on top, Python libraries underneath

Your test .robot file · readable steps	`Login As  standard_user`
Your keywords built from library keywords	`Fill Text, Click, Get Text`
Libraries Python · talk to the real browser or app	`Browser (Playwright), AutoIt…`

An **open-source automation framework written in Python**. You write tests as keywords; libraries supply the code behind them.

THE KEY IDEA

Robot Framework doesn’t know how to click a button.

Libraries do the work

RF orchestrates: parses files, calls keywords, writes reports

You compose keywords into readable tests

Settings and Variables come first

```
*** Settings ***
Documentation     What this suite is about
Library           Browser    # import a library
Resource          ../resources/common.resource    # shared keywords/variables
Suite Setup       Open Test Browser    # once before all tests
Suite Teardown    Close Browser    # once after all tests
Test Setup        Open Fresh Page    # before each test
Test Teardown     Close Context    # after each test

*** Variables ***
${URL}            https://www.saucedemo.com    # scalar
@{USERS}          standard_user    problem_user    # list
&{CREDS}          user=standard_user    pass=secret_sauce    # dictionary
```

Then Test Cases and Keywords

```
*** Test Cases ***
User Can Log In
    [Tags]    smoke
    Login As    standard_user    secret_sauce
    Get Text    .title    ==    Products

*** Keywords ***
Login As
    [Arguments]    ${username}    ${password}
    Fill Text      id=user-name    ${username}
    Fill Text      id=password     ${password}
    Click          id=login-button
```

Test Cases

Each test is a name followed by indented steps.

Keywords

Your own keywords, built from library keywords. [Arguments] names the inputs.

The syntax rules behind most beginner errors

Two or more spaces
separate keywords and arguments. Four is the convention.

One space is part of a name
Fill Text is one keyword.

Indent the steps
under the test or keyword name.

Variable	Holds
${scalar}	One value
@{list}	Several values
&{dict}	key=value pairs
%{HOME}	An environment variable

3 · SETUP IN VS CODE

Install the tools from the terminal

1 Prerequisites

Python 3.9+ and Node.js 18+. The Browser library uses Playwright, which runs on Node.

```
python --version
node --version
```

2 Project and virtual environment

Keeps this project's packages separate.

```
mkdir saucedemo-robot && cd saucedemo-robot
python -m venv .venv
# Windows:      .venv\Scripts\activate
# macOS/Linux:  source .venv/bin/activate
```

3 Robot Framework + Browser

rfbrowser init downloads Playwright and the browsers — it takes a few minutes.

```
pip install robotframework robotframework-browser
rfbrowser init
```

3 · SETUP IN VS CODE

Then point VS Code at the project

4

Install RobotCode

Publisher: Daniel Biehl. Syntax highlighting, keyword autocompletion, go-to-definition, a test explorer with run and debug, and breakpoints in .robot files.

5

Select the interpreter

Ctrl+Shift+P → “Python: Select Interpreter” → pick .venv.

Red squiggles on keywords? The extension usually isn’t using the venv with the libraries.

6

Add robot.toml

In the project root. RobotCode reads it, so every run puts results in one folder.

```
output-dir = "results"
```

4 · PROJECT STRUCTURE

One resource file per page

```
saucedemo-robot/
├── robot.toml
├── requirements.txt
├── resources/
│   ├── common.resource     # browser setup, shared vars
│   ├── login_page.resource
│   ├── inventory_page.resource
│   └── checkout_page.resource
├── tests/
│   ├── login.robot
│   └── checkout.robot
└── desktop/
    └── notepad_autoit.robot     # Windows only
```

Resource files hold keywords and variables — no tests.

Split per page, so when Sauce Demo changes a locator you fix it in **one place**.

Tests live in tests/ and import what they need.

5 · SAUCE DEMO · RESOURCES/COMMON.RESOURCE

A practice shop and the shared setup

Sauce Labs' practice site **saucedemo.com**. Test users, all with password secret_sauce:

- `standard_user`
- `locked_out_user`
- `problem_user`
- `performance_glitch_user`

```
*** Settings ***
Library    Browser

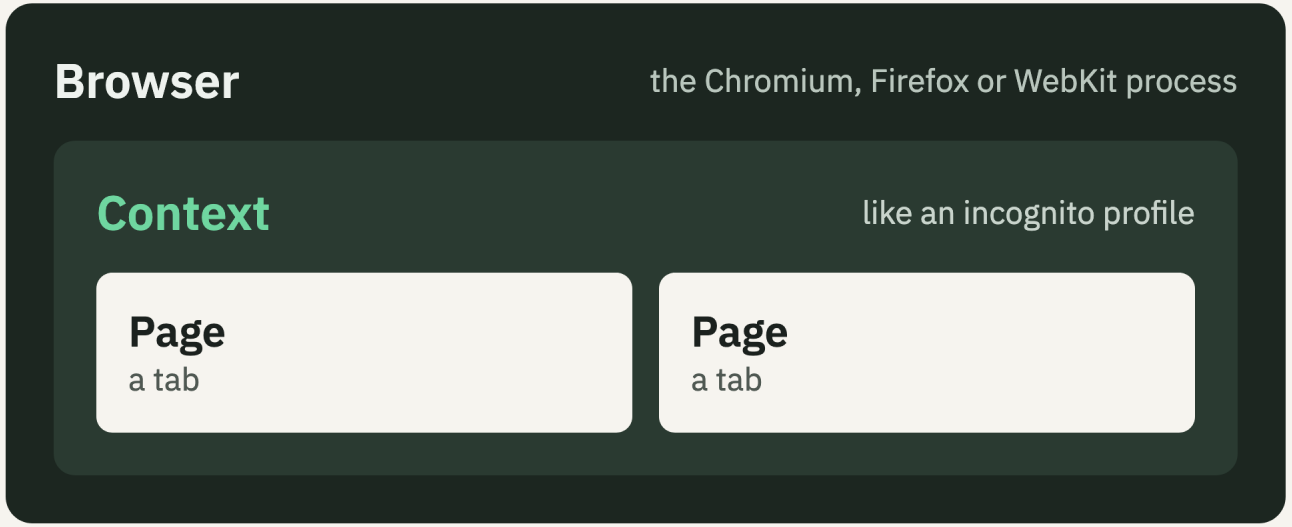
*** Variables ***
${URL}          https://www.saucedemo.com
${BROWSER}      chromium
${HEADLESS}     False
${PASSWORD}     secret_sauce

*** Keywords ***
Open Test Browser
    New Browser    ${BROWSER}    headless=${HEADLESS}

Open Fresh Page
    New Context    viewport={'width': 1280, 'height': 800}
    New Page       ${URL}
```

5 · BROWSER LIBRARY

Browser, Context, Page: three levels



One browser per suite, a new context per test.

Each context has its own cookies and storage, so every test gets a clean session — without relaunching the browser.

5 · RESOURCES/LOGIN_PAGE.RESOURCE

Locators as variables, assertions built in

```
*** Settings ***
Library    Browser

*** Variables ***
${USERNAME_FIELD}    id=user-name
${PASSWORD_FIELD}    id=password
${LOGIN_BUTTON}      id=login-button
${ERROR_MESSAGE}     [data-test="error"]

*** Keywords ***
Login As
    [Arguments]    ${username}    ${password}
    Fill Text      ${USERNAME_FIELD}    ${username}
    Fill Text      ${PASSWORD_FIELD}    ${password}
    Click          ${LOGIN_BUTTON}

Login Should Fail With
    [Arguments]    ${expected}
    Get Text       ${ERROR_MESSAGE}    *=    ${expected}
```

Get Text *selector operator value*

`==`	equals
`*=`	contains
`^=`	starts with

Assertions auto-wait and retry until a timeout — no “sleep” hacks like in Selenium.

5 · RESOURCES/INVENTORY_PAGE.RESOURCE

The product list: add, sort, check

```
*** Settings ***
Library    Browser

*** Keywords ***
Inventory Page Should Be Open
    Get Text    .title    ==    Products
    Get Url    *=    inventory.html

Add Product To Cart
    [Arguments]    ${product_id}
    Click    id=add-to-cart-${product_id}

Cart Badge Should Show
    [Arguments]    ${count}
    Get Text    .shopping_cart_badge    ==    ${count}
```

```
Sort Products By
    [Arguments]    ${option}
    # az, za, lohi, hilo
    Select Options By
    ...    [data-test="product-sort-container"]
    ...    value    ${option}

First Product Price Should Be
    [Arguments]    ${price}
    Get Text    .inventory_item_price >> nth=0
    ...    ==    ${price}

Open Cart
    Click    .shopping_cart_link
```

5 · RESOURCES/CHECKOUT_PAGE.RESOURCE

Checkout: fill the form, finish, confirm

```
*** Settings ***
Library    Browser

*** Keywords ***
Checkout With Details
    [Arguments]    ${first}    ${last}    ${zip}
    Click          id=checkout
    Fill Text      id=first-name      ${first}
    Fill Text      id=last-name       ${last}
    Fill Text      id=postal-code     ${zip}
    Click          id=continue

Finish Order
    Click    id=finish

Order Should Be Complete
    Get Text    .complete-header    ==    Thank you for
your order!
```

Small keywords, one job each.

Actions (*Checkout With Details*, *Finish Order*) are kept apart from checks (*Order Should Be Complete*).

Tests can then stop between steps and verify the totals.

5 · TESTS/LOGIN.ROBOT

A test file imports what it needs

```
*** Settings ***
Documentation     Login tests for Sauce Demo
Resource          ../resources/common.resource
Resource          ../resources/login_page.resource
Resource          ../resources/inventory_page.resource
Suite Setup       Open Test Browser
Suite Teardown    Close Browser
Test Setup        Open Fresh Page
Test Teardown     Close Context

*** Test Cases ***
Standard User Can Log In
    [Tags]    smoke
    Login As    standard_user    ${PASSWORD}
    Inventory Page Should Be Open
```

The test is two readable lines.

Suite Setup opens the browser once. Test Setup gives each test a fresh context and page.

${PASSWORD} comes from common.resource.

[Template] turns one keyword into a table

```
Invalid Logins Show Correct Errors
    [Tags]    negative
    [Template]    Invalid Login Attempt
    # username          password        expected error
    locked_out_user     ${PASSWORD}     Sorry, this user has been locked out
    ${EMPTY}            ${PASSWORD}     Username is required
    standard_user       ${EMPTY}        Password is required
    wrong_user          wrong_pass      do not match any user

*** Keywords ***
Invalid Login Attempt
    [Arguments]    ${username}    ${password}    ${error}
    Login As    ${username}    ${password}
    Login Should Fail With    ${error}
```

Each row runs as one iteration. ${EMPTY} is a built-in empty string.

5 · TESTS/CHECKOUT.ROBOT · END TO END

A full purchase, built from page keywords

```
*** Test Cases ***
User Can Buy Two Products
    [Tags]    smoke    e2e
    Add Product To Cart    sauce-labs-backpack
    Add Product To Cart    sauce-labs-bike-light
    Cart Badge Should Show    2
    Open Cart
    Get Element Count    .cart_item    ==    2
    Checkout With Details    Fred    Tester    SW1A1AA
    Get Text    .summary_subtotal_label    *=    39.98
    Finish Order
    Order Should Be Complete
```

```
# in *** Settings ***
Test Setup    Login As Standard User

Sort By Price Low To High
    Sort Products By    lohi
    First Product Price Should Be
    ...    $7.99

*** Keywords ***
Login As Standard User
    Open Fresh Page
    Login As    standard_user    ${PASSWORD}
```

RUNNING THE TESTS · VENV ACTIVE

Run everything, or just what you need

`robot -d results tests/`	Run everything
`robot -d results --include smoke tests/`	Only tests tagged smoke
`robot -d results --test "Sort By*" tests/`	By name, wildcards OK
`robot -d results -v HEADLESS:True -v BROWSER:firefox tests/`	Override variables

In VS Code: click ▶ next to any test, or set a breakpoint and choose **Debug Test**.

READING THE RESULTS

Every run writes three files to results/

`report.html`

The high-level pass/fail summary.

`log.html`

The most useful. Every keyword, its arguments, timings, and screenshots on failure.

`output.xml`

Machine-readable results for CI tools such as Jenkins.

FOR CI Set HEADLESS:True. The official Docker image marketsquare/robotframework-browser has the browsers preinstalled.

ROBOT FRAMEWORK · BONUS AFTER SECTION 10

Test like you talk

Writing scenarios with Given / When / Then — and breaking them down into variables, keywords and templates

`Given` · `When` · `Then` · `And` · `But`

THE METHOD

Four steps from plain English to a running test

1

Plain English

Describe the behaviour the way a user or business person would. No code yet.

2

Data → variables

Mark every concrete value in the scenarios. Each one is a candidate variable.

3

Sentences → keywords

Every Given, When and Then line becomes a keyword.

4

Repeats → templates

Same steps with different data? Write it once, list the data as rows.

Then **write the test files** and run them.

STEP 1 · PLAIN ENGLISH

Start with no code at all

```
Scenario 1: Successful purchase
  Given the user is logged in as a standard user
  When the user adds the Backpack to the cart
  And the user adds the Bike Light to the cart
  And the user checks out as Fred Tester
      with postcode SW1A 1AA
  Then the order total should be $43.18
  And the order should be confirmed
```

Describe the behaviour the way a user or business person would.

Anyone on the team should be able to read it — and agree it describes what the shop must do.

STEP 1 · PLAIN ENGLISH

Each word has one job

Given

Sets up the starting state

the user is logged in as a standard user

When

The action

the user adds the Backpack to the cart

Then

The check

the order total should be $43.18

And / But

Continue the previous step type

And the order should be confirmed — still a Then

STEP 1 · PLAIN ENGLISH

Four scenarios for Sauce Demo

Scenario	What it checks	Shape
1 · Successful purchase	Two products, checkout, total $43.18, confirmation	One-off
2 · Login is rejected for bad credentials	Locked out, empty username, empty password, wrong credentials	Repeats with new data
3 · Products can be sorted	The first product for each sort option	Repeats with new data
4 · Removing a product from the cart	The cart is empty after removing the Backpack	One-off

Keep an eye on the ones that repeat — they become templates in Step 4.

STEP 2 · THE DATA

Mark every concrete value

Value in scenario	Variable	Why
standard user / secret_sauce	${STANDARD_USER}, ${PASSWORD}	Reused everywhere
Backpack, Bike Light	&{PRODUCTS} dictionary	Maps friendly names to the site's IDs
Fred, Tester, SW1A 1AA	&{CUSTOMER} dictionary	Grouped checkout details
$43.18	${EXPECTED_TOTAL}	Expected result
Sort option labels	&{SORT_OPTIONS}	Maps "Price (low to high)" to lohi
Error messages, usernames	Template rows	They vary per case, so they become template data

RULE OF THUMB

> **Used in many places, or likely to change between environments?**
>
> → `a variable`

> **Different in each repetition of the same scenario?**
>
> → `template data`

STEP 2 · THE DATA

One file holds all the data

```
*** Variables ***
${URL}                https://www.saucedemo.com
${STANDARD_USER}      standard_user
${PASSWORD}           secret_sauce

&{PRODUCTS}
...     Backpack=sauce-labs-backpack
...     Bike Light=sauce-labs-bike-light
...     Onesie=sauce-labs-onesie
...     Fleece Jacket=sauce-labs-fleece-jacket

&{CUSTOMER}     first=Fred     last=Tester     postcode=SW1A 1AA

&{SORT_OPTIONS}
...     Name (A to Z)=az
...     Name (Z to A)=za
...     Price (low to high)=lohi
...     Price (high to low)=hilo
```

resources/
variables.resource

Dictionaries let scenarios use human words — **“Bike Light”** — while keywords look up the technical value: sauce-labs-bike-light.

The **...** continues the previous line.

STEP 3 · KEYWORDS

Every sentence becomes a keyword

Scenario sentence	Keyword
the user is logged in as a standard user	The User Is Logged In As A Standard User
the user adds the Backpack to the cart	The User Adds The "${product}" To The Cart
the user checks out as Fred Tester…	The User Checks Out With Customer Details
the order total should be $43.18	The Order Total Should Be "${total}"
the user sorts products by “…”	The User Sorts Products By "${option}"
the first product should be “…”	The First Product Should Be "${name}"

Where a sentence holds data, use an **embedded argument** so one keyword covers many sentences.

STEP 3 · KEYWORDS

Leave Given and When out of the name

Robot strips the prefix when matching, so both of these lines call the **same** keyword:

```
Given The User Is Logged In As A Standard User
And   The User Is Logged In As A Standard User
```

The keyword is defined once as The User Is Logged In As A Standard User

STEP 3 · KEYWORDS

Separate what a step means from how it clicks

Test case tests/*.robot	`Given The User Is Logged In As A Standard User`
Step keyword steps.resource · *what*	`Login As  ${STANDARD_USER}  ${PASSWORD}`
Page keyword pages.resource · *how*	`Fill Text, Click on the login page`
Library keyword Browser library	`Playwright drives the browser`

If Sauce Demo changes a button ID, **only the page layer changes**. Your scenarios stay the same.

STEP 3 · RESOURCES/PAGES.RESOURCE

The page layer: locators and clicks

```
*** Settings ***
Library     Browser
Resource    variables.resource

*** Keywords ***
Open Login Page
    New Context
    ...    viewport={'width': 1280, 'height': 800}
    New Page       ${URL}

Login As
    [Arguments]    ${username}    ${password}
    Fill Text    id=user-name    ${username}
    Fill Text    id=password     ${password}
    Click        id=login-button
```

```
Add To Cart
    [Arguments]    ${product_id}
    Click    id=add-to-cart-${product_id}

Remove From Cart
    [Arguments]    ${product_id}
    Click    id=remove-${product_id}

Go To Cart
    Click    .shopping_cart_link

Fill Checkout Form
    [Arguments]    ${first}    ${last}    ${postcode}
    Click        id=checkout
    Fill Text    id=first-name     ${first}
    Fill Text    id=last-name      ${last}
    Fill Text    id=postal-code    ${postcode}
    Click        id=continue
```

STEP 3 · RESOURCES/STEPS.RESOURCE

The step layer: Given keywords

```
*** Settings ***
Library     Browser
Resource    variables.resource
Resource    pages.resource

*** Keywords ***
# ---------- GIVEN ----------
The User Is On The Login Page
    Open Login Page

The User Is Logged In As A Standard User
    Open Login Page
    Login As    ${STANDARD_USER}    ${PASSWORD}
    Get Text    .title    ==    Products

The User Is Logged In With The "${product}" In The Cart
    The User Is Logged In As A Standard User
    The User Adds The "${product}" To The Cart
```

Steps speak the scenario language.

They call page keywords — and each other. The last Given reuses two existing steps.

Get Text ... == checks we really landed on Products.

STEP 3 · RESOURCES/STEPS.RESOURCE

The step layer: When keywords

```
The User Logs In As "${username}" With Password "${password}"
    Login As    ${username}    ${password}

The User Adds The "${product}" To The Cart
    Add To Cart    ${PRODUCTS}[${product}]

The User Removes The "${product}"
    Go To Cart
    Remove From Cart    ${PRODUCTS}[${product}]

The User Checks Out With Customer Details
    Go To Cart
    Fill Checkout Form    ${CUSTOMER.first}    ${CUSTOMER.last}
    ...    ${CUSTOMER.postcode}

The User Sorts Products By "${option}"
    Select Options By    [data-test="product-sort-container"]
    ...    value    ${SORT_OPTIONS}[${option}]
```

Human words in, technical values out.

${PRODUCTS}[${product}] turns “Backpack” into sauce-labs-backpack.

${CUSTOMER.first} reads one item from the dictionary.

STEP 3 · RESOURCES/STEPS.RESOURCE

The step layer: Then keywords

```
# ---------- THEN ----------
The Error "${message}" Should Be Shown
    Get Text    [data-test="error"]    *=    ${message}

The Order Total Should Be "${total}"
    Get Text    .summary_total_label    *=    ${total}

The Order Should Be Confirmed
    Click       id=finish
    Get Text    .complete-header    ==    Thank you for your order!

The First Product Should Be "${name}"
    Get Text    .inventory_item_name >> nth=0    ==    ${name}

The Cart Should Be Empty
    Get Element Count    .cart_item    ==    0
```

Then keywords are the checks.

== the text must match exactly

*= the text must contain the value

STEP 4 · TEMPLATES

Same steps, different data: write it once

1 Write the scenario once as a keyword whose arguments are the parts that change.

2 List each set of data as a row. Robot fills the ${...} slots from each row's columns, in order.

```
# ---------- SCENARIO TEMPLATES ---------- (add to steps.resource)
Login With "${username}" And "${password}" Should Show "${error}"
    Given The User Is On The Login Page
    When The User Logs In As "${username}" With Password "${password}"
    Then The Error "${error}" Should Be Shown

Sorting By "${option}" Should Show "${first_product}" First
    Given The User Is Logged In As A Standard User
    When The User Sorts Products By "${option}"
    Then The First Product Should Be "${first_product}"
```

STEP 5 · TEST FILES

Every test file starts the same way

```
*** Settings ***
Documentation     Shopping scenarios for Sauce Demo
Resource          ../resources/steps.resource
Suite Setup       New Browser    chromium    headless=False
Suite Teardown    Close Browser
Test Teardown     Close Context
Test Tags         shopping
```

Resource

One import brings in steps, pages and variables.

Suite Setup

One browser for the whole file, closed at the end.

Test Teardown

A fresh context per test — no leftover cart or login.

Test Tags

Label tests so you can run just a subset.

STEP 5 · TESTS/PURCHASE.ROBOT

The tests read like the scenarios

```
*** Test Cases ***
Successful Purchase Of Two Products
    [Tags]    smoke    checkout
    Given The User Is Logged In As A Standard User
    When The User Adds The "Backpack" To The Cart
    And The User Adds The "Bike Light" To The Cart
    And The User Checks Out With Customer Details
    Then The Order Total Should Be "$43.18"
    And The Order Should Be Confirmed

Removing A Product Empties The Cart
    [Tags]    cart
    Given The User Is Logged In With The "Backpack" In The Cart
    When The User Removes The "Backpack"
    Then The Cart Should Be Empty
```

Compare it with Scenario 1 — almost word for word.

No locators, no IDs. All of that lives in the resource files.

[Tags] adds to the file's Test Tags.

STEP 5 · TESTS/LOGIN.ROBOT

One template, one test per row

```
# in *** Settings ***, alongside the shared lines
Test Template     Login With "${username}" And "${password}" Should Show "${error}"
Test Tags         login    negative
```

*** Test Cases ***	USERNAME	PASSWORD	ERROR
Locked Out User	locked_out_user	secret_sauce	Sorry, this user has been locked out
Missing Username	${EMPTY}	secret_sauce	Username is required
Missing Password	standard_user	${EMPTY}	Password is required
Wrong Credentials	wrong_user	wrong_pass	do not match any user

Each row is its own test case. ${EMPTY} is Robot’s built-in empty string.

STEP 5 · TESTS/SORTING.ROBOT

Four sort options, four rows

```
# in *** Settings ***, alongside the shared lines
Test Template     Sorting By "${option}" Should Show "${first_product}" First
Test Tags         sorting
```

*** Test Cases ***	OPTION	FIRST PRODUCT
Sort Name A To Z	Name (A to Z)	Sauce Labs Backpack
Sort Name Z To A	Name (Z to A)	Test.allTheThings() T-Shirt (Red)
Sort Price Low To High	Price (low to high)	Sauce Labs Onesie
Sort Price High To Low	Price (high to low)	Sauce Labs Fleece Jacket

STEP 5 · RUN THEM

Tags pick which scenarios run

```
robot -d results tests/
```
All scenarios

```
robot -d results --include smoke tests/
```
Just the purchase smoke test

```
robot -d results --include negative tests/
```
Just the login errors

SUMMARY

Four questions break down any scenario

Question to ask	Becomes
What values appear? Are they reused or environment-specific?	Variables in variables.resource
What does each sentence mean?	Step keywords with embedded arguments, in steps.resource
How does the browser actually do it?	Page keywords with locators, in pages.resource
Is the same scenario repeated with different data?	Templates: one keyword, many rows

BEGINNER EXERCISE

Your turn: break this one down

```
Scenario: Checkout requires a postcode
  Given the user is logged in
    with the Onesie in the cart
  When the user checks out
    without a postcode
  Then the error "Postal Code is required"
    should be shown

(repeat for missing first name
 and missing last name)
```

1. Which values become variables, and which become template data?
2. Which existing keywords can you reuse? *Hint: two of the three steps already exist.*
3. Write the new keyword and a Test Template file with three rows.